Multi-threading

CS 2410

Wonsun Ahn

University of Pittsburgh

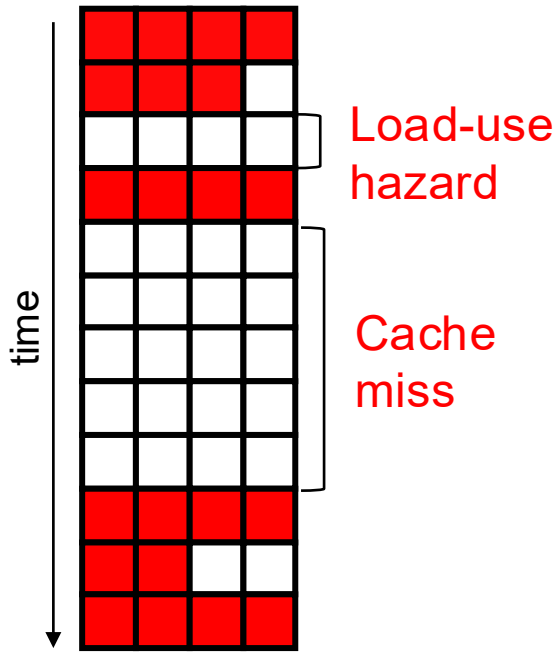
Multi-threading (MT)

- Even moderate superscalars (e.g., 4-way) are not fully utilized
  - Average sustained IPC: 1.5–2 → < 50% utilization due to
    - Control dependencies (mis-predicted branches)
    - Data dependences
    - Cache misses

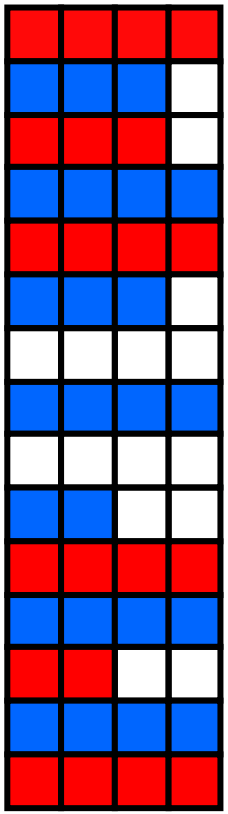
- Multi-threading (MT) to the rescue
  - **Improves utilization** by multiplexing multiple threads (processes)
  - If one thread cannot fully utilize the datapath, maybe 2 or 4 (or 100) can

Multithreading Example

- Time evolution of issue slots
  - 4-way datapath



Static SS



Multithreaded

- # of wasted slots?

- In this example, we have 2 threads that take turns issuing every cycle
  - Called fine-grained multi-threading
- Some observations:
  - Removes load-use hazard bubbles
  - Partially removes cache miss bubbles
  - Takes longer for "red" thread to finish
    - With more threads, would take even longer
  - Still some bubbles (but stay tuned)

University of Pittsburgh

MT trades off latency for throughput

- MT trades (single-thread) latency for throughput
  - Degrades latency of individual threads
  - + But improves throughput via increased utilization (A multi-threaded program will finish faster)

- Different workloads have different types of parallelism
  - SpecFP (Floating Point benchmarks in SpecCPU) has lots of ILP
    - Mainly comprised of data parallel loops
  - Server workloads have TLP (Thread Level Parallelism)
    - Often complex control/data flow with limited ILP
    - But multi-threaded with a thread handling each requst

Three Types of MT Implementations

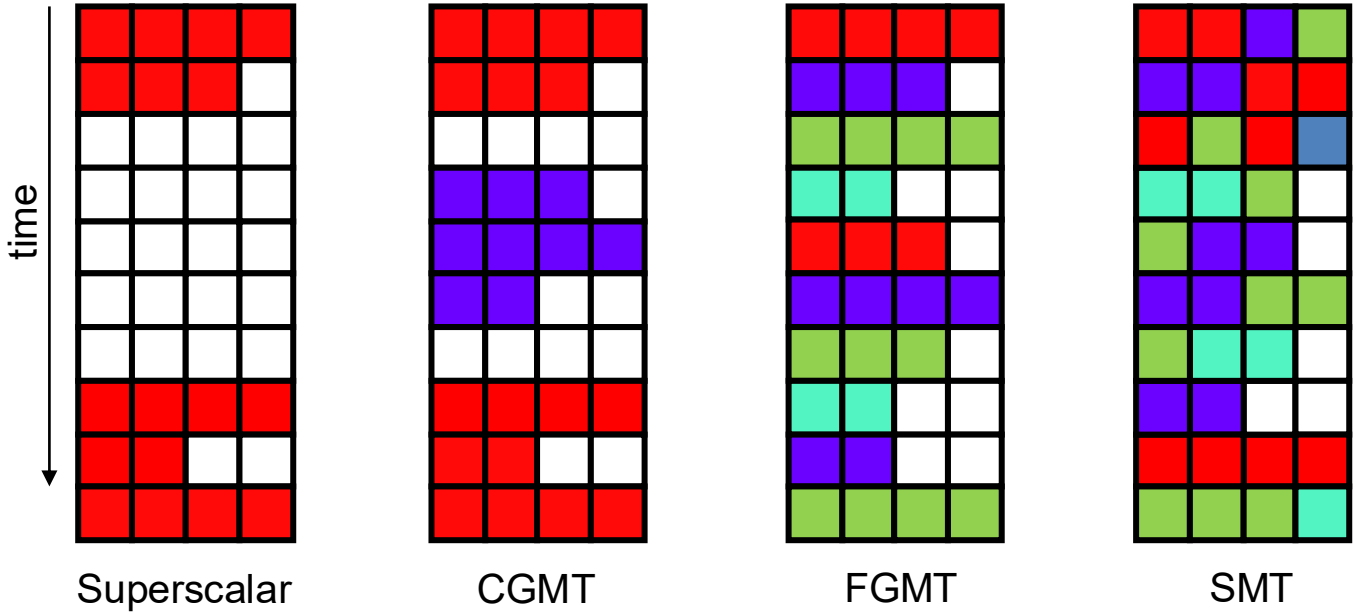
- Categorized by: **thread scheduling granularity**
  - How frequently to switch issuing from one thread to another?
  - How do threads share issue slots of the superscalar?

- Three types:
  - Coarse-grain multithreading (**CGMT**)
  - Fine-grain multithreading (**FGMT**)
  - Simultaneous multithreading (**SMT**)

- Choices depends on:
  - What kind of latencies you want to hide
  - How much single thread performance you are willing to sacrifice

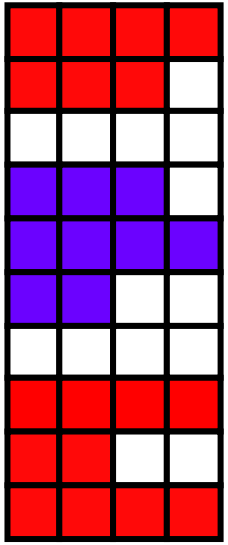
Three Types of MT Implementations (Visualized)

- Time evolution of issue slots
  - Color = thread



Coarse-Grain Multi-Threading (CGMT)

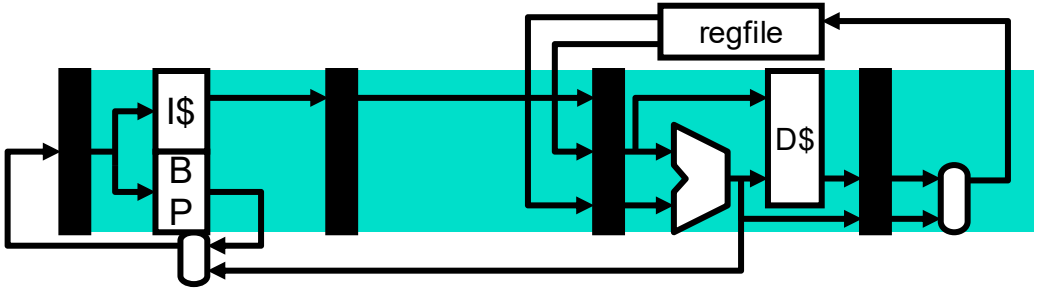
- Thread scheduling policy
  - One primary thread and one or more backup threads
  - Switch primary to backup only on a long latency hazard (e.g. a cache miss that needs to go to DRAM memory)
  - By the time primary thread executes, hazard is resolved
- Issue slot partitioning
  - Issues slots in same cycle cannot be shared
  - Flush on switch to prevent insts from getting mixed up
  - ☞So can't hide very short latencies
- Example: IBM Northstar / Intel Itanium 2



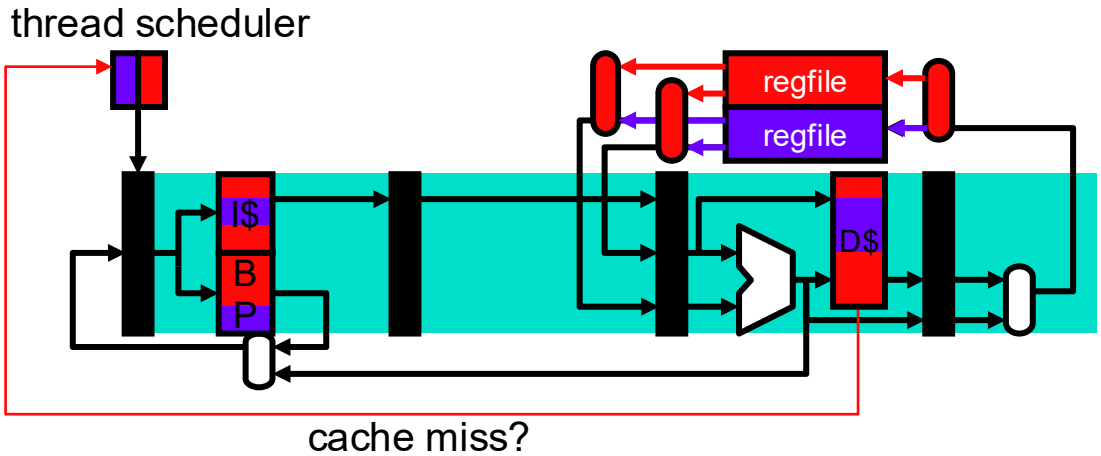
CGMT

+ Sacrifices little single-thread performance (except flush penalty)
– Can hide only long latencies (e.g., accesses to DRAM)

Coarse-Grain Multi-Threading Architecture

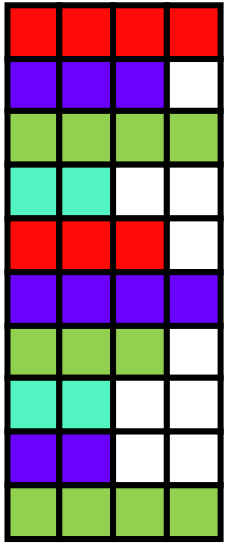


CGMT



Fine-Grain Multi-Threading (FGMT)

- Thread scheduling policy
  - Switch threads every cycle round-robin
- Issue slot partitioning
  - Issues slots in same cycle cannot be shared
  - No pipeline flushing between threads
  - ☞So can hide short latencies
- Example: Sun's UltraSPARC T1 (Niagara)



FGMT

+ Hides short latencies (e.g. load-use hazards, mis-predicted branches)
– Sacrifices single thread performance (proportional to thread num)

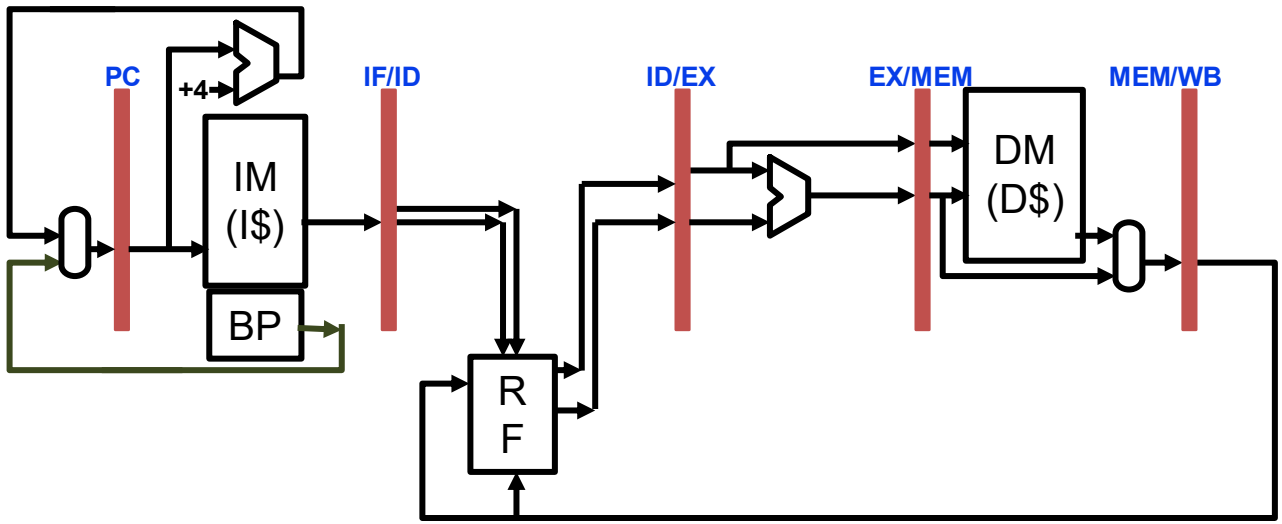
FGMT Sharing Implementations Issues

- How does FGMT allow insts from multiple threads to co-exist?
  - Must maintain state for each concurrent thread

- No state: ALUs
  - So, can be dynamically shared
- Persistent hard state (a.k.a thread "context"): PC, Register File
  - So must be **replicated**
- Persistent soft state: Caches, branch prediction tables, TLBs
  - Dynamically partitioned (like on a multi-programmed processor)
- Transient state: pipeline latches
  - Must be partitioned … somehow

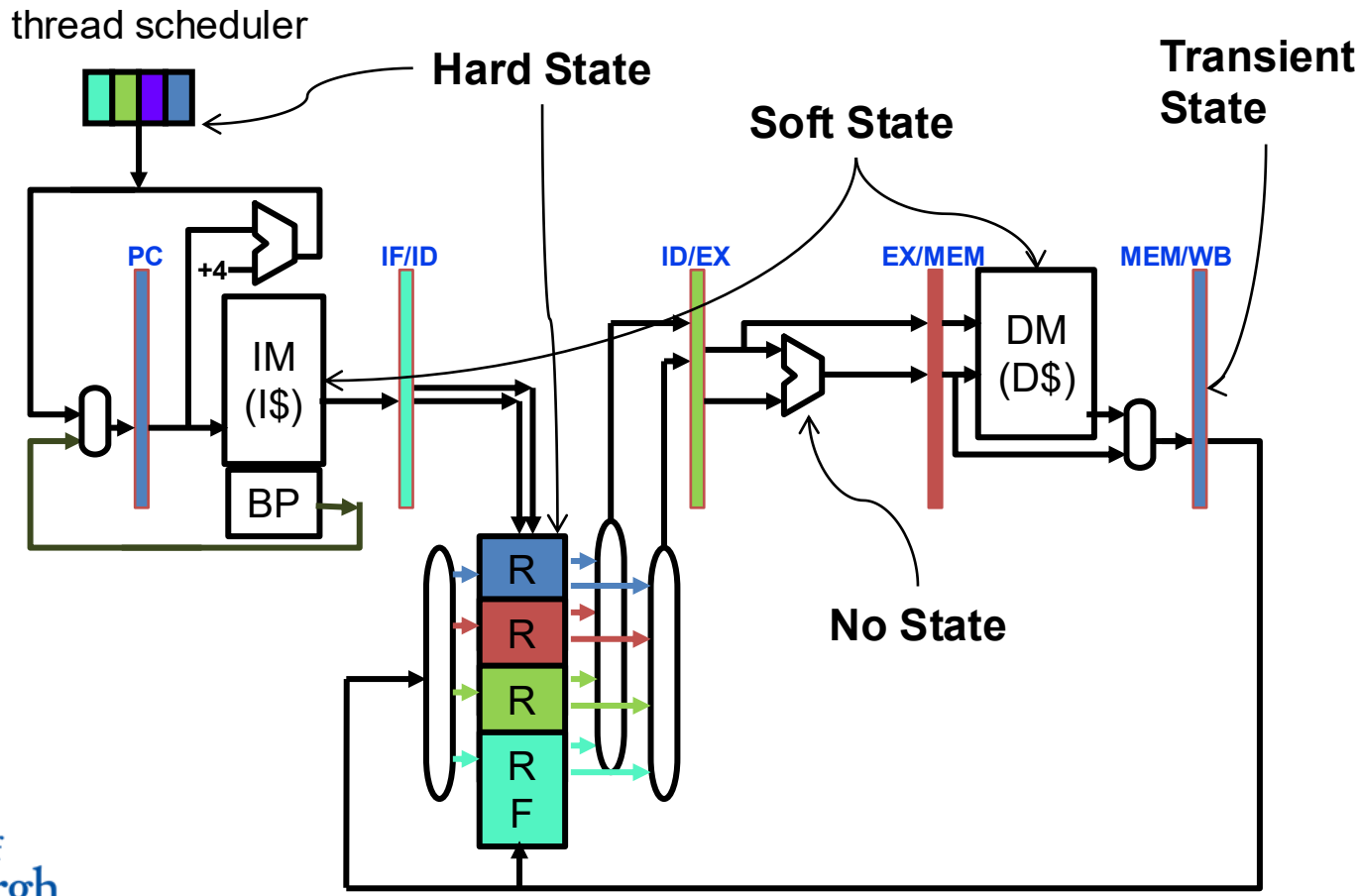
FGMT Architecture (Single Issue)

- What do we have to add to our datapath to support FGMT?



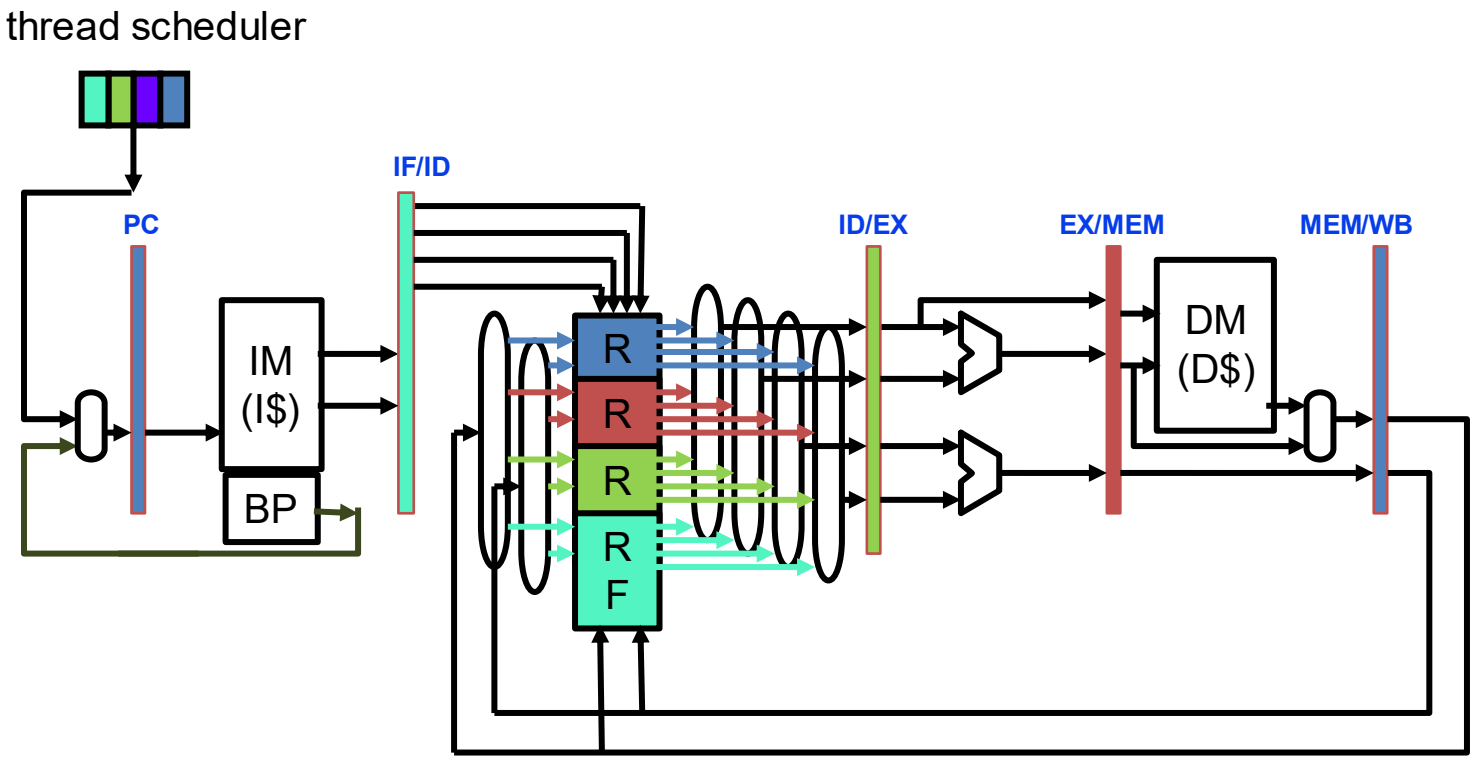
FGMT Architecture (Single Issue)

- What do we have to add to our datapath to support FGMT?



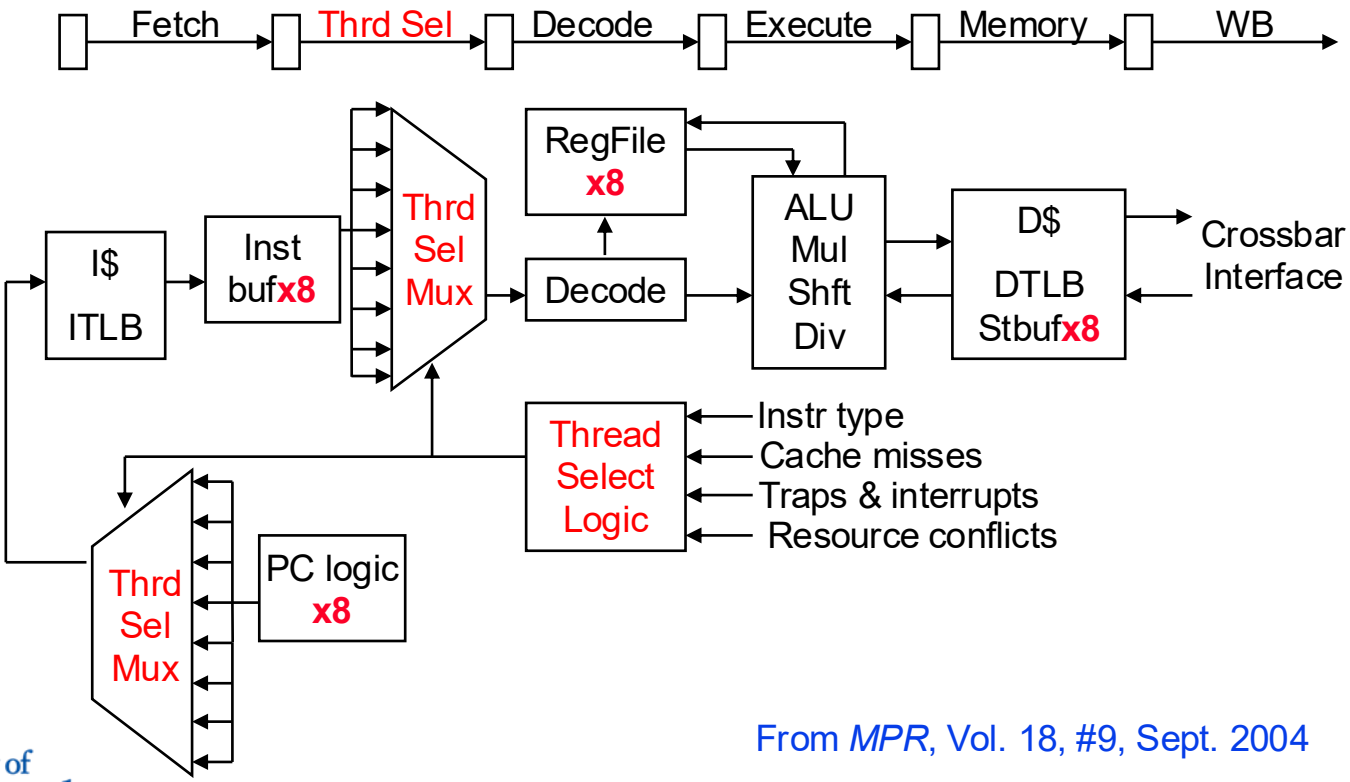
FGMT Architecture (2-Way Issue)

- What do we have to add to our datapath to support FGMT?



Sun Niagara's FGMT Integer pipeline

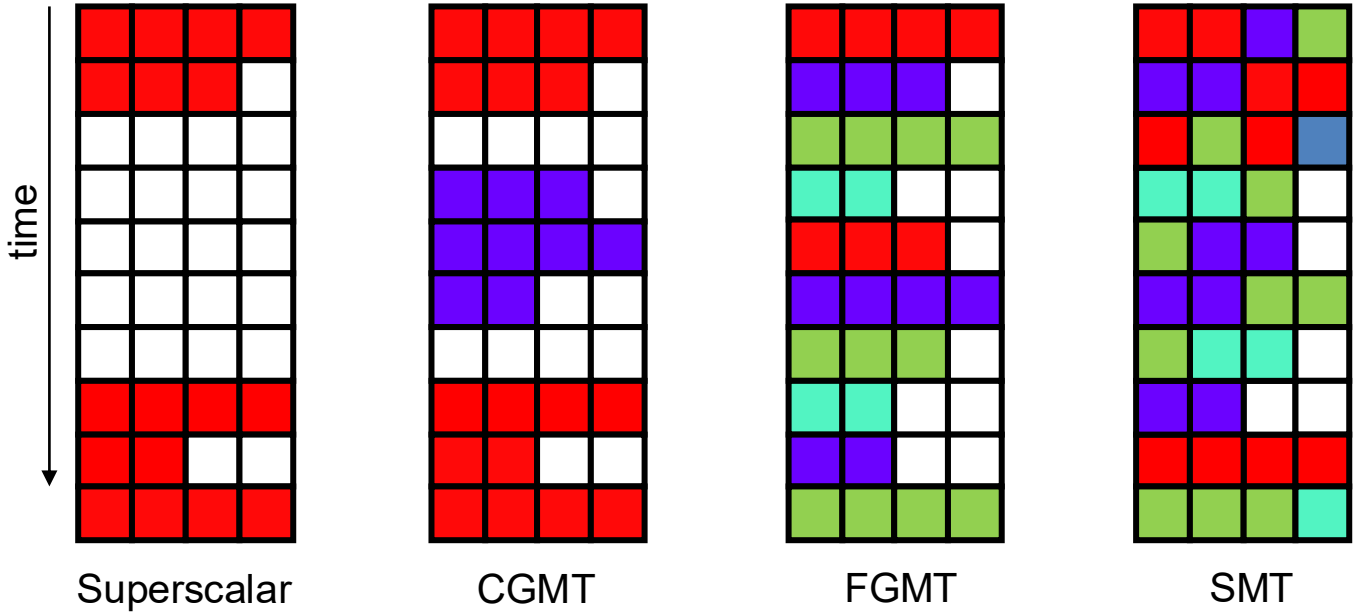
- Cores are simple (single-issue, in-order 6 stage pipeline, no branch prediction), small, and power-efficient



From *MPR*, Vol. 18, #9, Sept. 2004

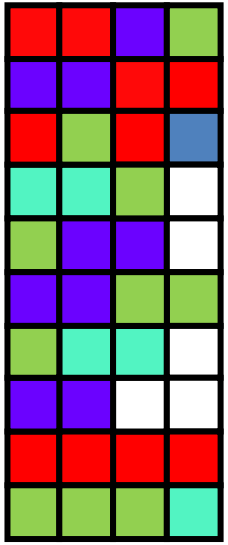
Three Types of MT Implementations (Visualized)

- Time evolution of issue slots
  - Color = thread



Simultaneous Multi-Threading (SMT)

- Thread scheduling policy
  - All threads execute concurrently every cycle!
- Issue slot partitioning
  - Issue slots dynamically partitioned among threads
  - Works seamlessly with Out-of-Order superscalars (Once renamed, issue queue doesn't care which thread)
- Examples:
  - IBM Power7: 8-way, 4 threads
  - Intel Core i7: 4-way, 2 threads
  - AMD Bulldozer: 4-way, 2 threads
  - Alpha 21464: 8-way issue, 4 threads
  - Notice a pattern? #threads (T) * 2 = #issue width (N)

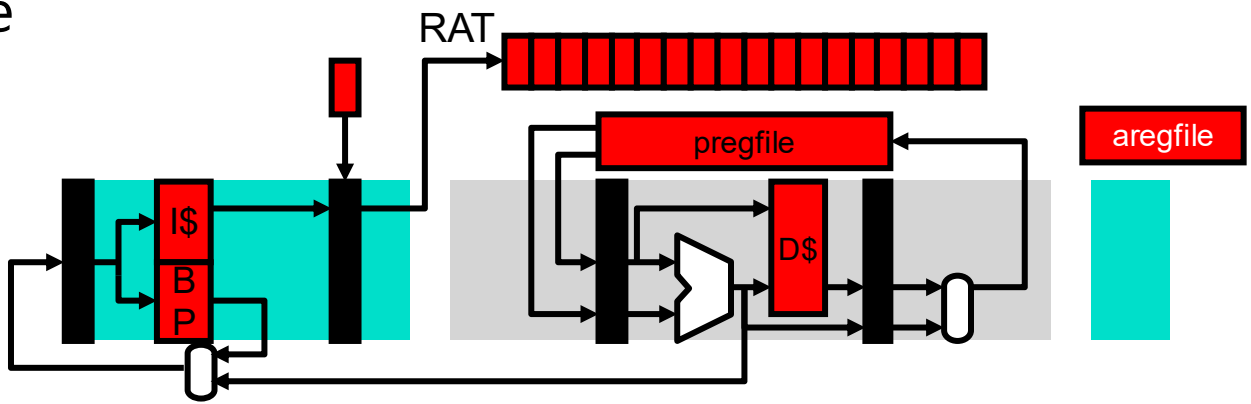


SMT

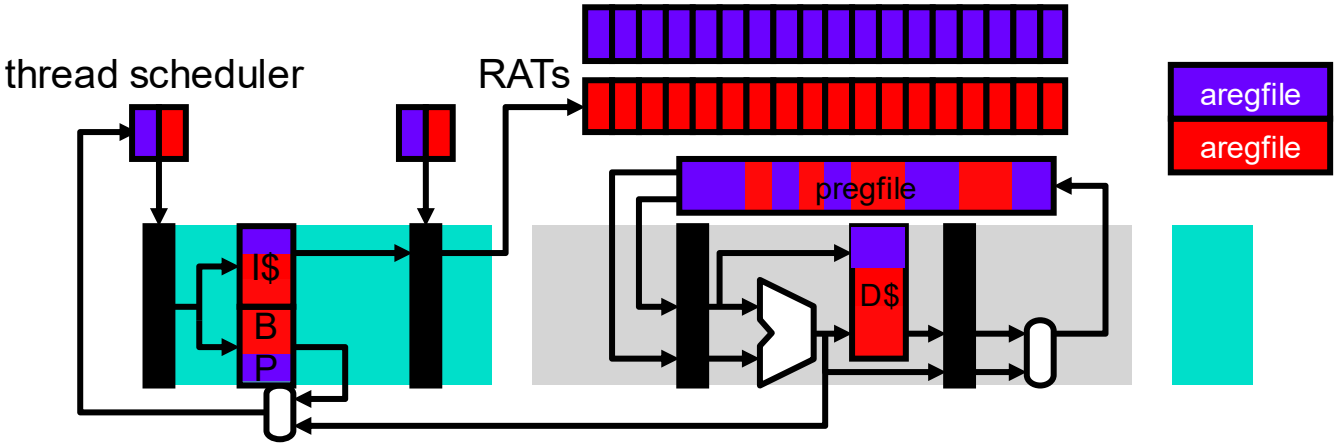
+ Fully exploits ILP and TLP at the same time
– Sacrifices single thread performance (proportional to thread num)

Simultaneous Multi-Threaded Architecture

- OOO Pipeline

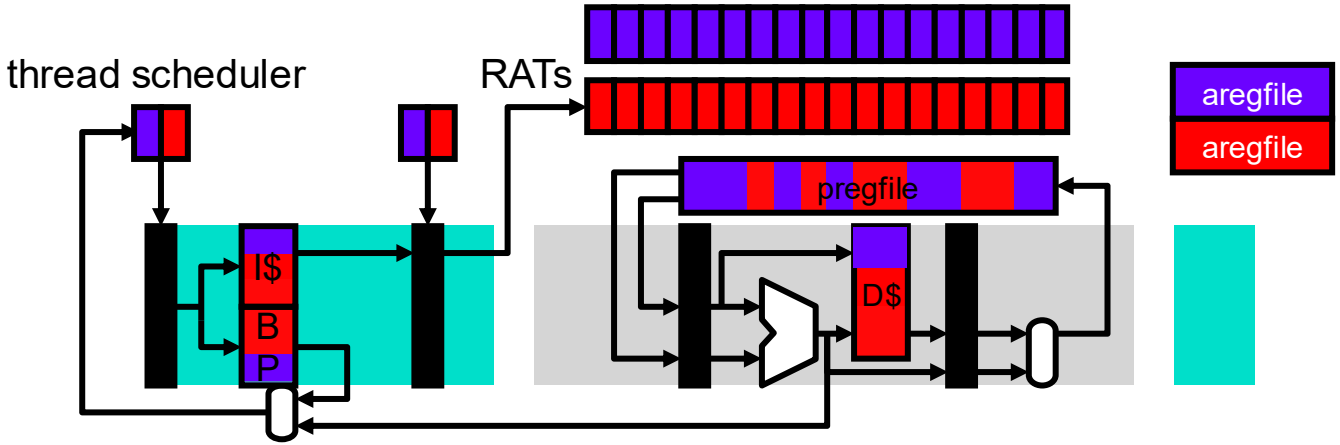


- SMT
  - Shared: physical register file, issue queue, icache, dcache
  - Duplicated: RAT, architectural register file

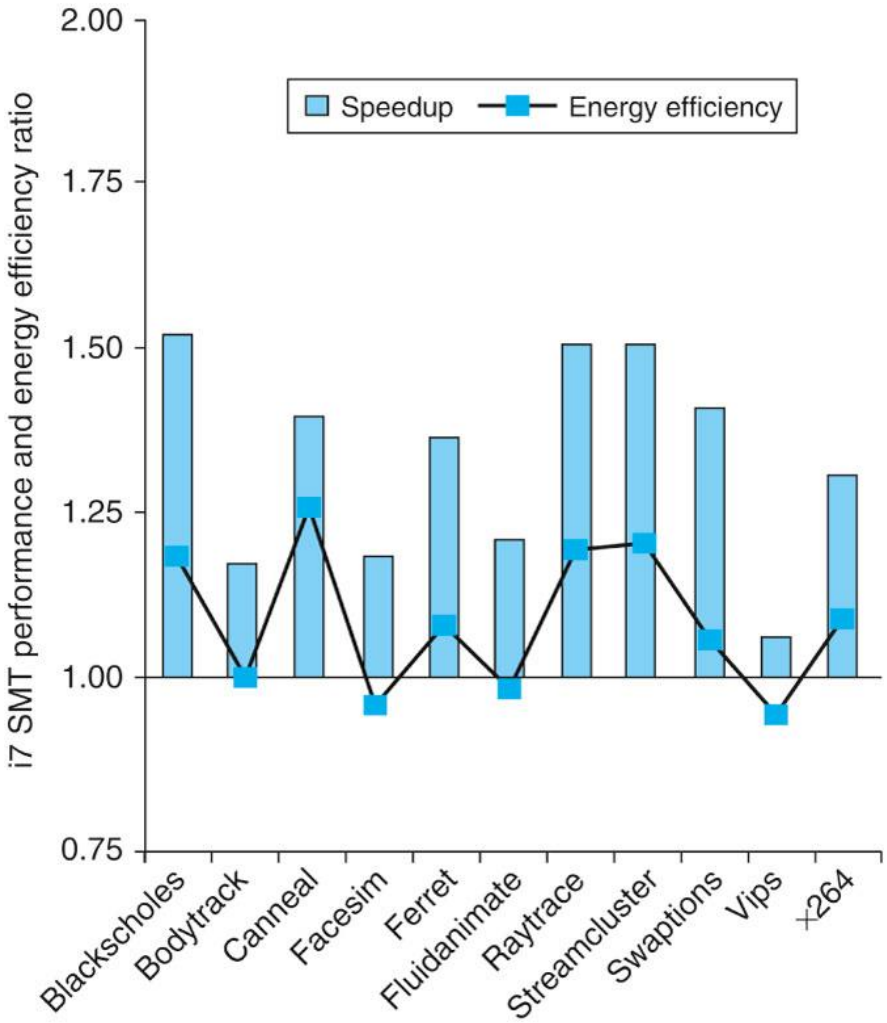


ROB & LSQ are partitioned among threads

- **Static partitioning**
  - ○ T equal-sized contiguous partitions
  - ± No starvation, sub-optimal utilization (fragmentation)
- **Dynamic partitioning**
  - ○ P > T partitions, available partitions assigned on need basis
  - ± Better utilization, possible starvation



Multithreading Speed-Ups on the i7 (SMT)



Multithreading vs Multicore

- If you wanted to run multiple threads would you build a
  - A multicore: multiple separate pipelines?
  - A multithreaded processor: a single larger pipeline?
- **Both will get you throughput on multiple threads**
  - A multicore will be simpler, possibly faster clock
    - Multicore is mainly a TLP (thread-level parallelism) engine
  - SMT will get you better performance (IPC) on a single core
    - SMT is basically an ILP engine that converts TLP to ILP
- **Do both**
  - Intel's Sandy (Ivy) Bridge and Haswell, IBM's Power7 & 8
  - 4~8 OOO 4-way cores each of which supports 2~4 threads (SMT)

A Review of Parallelism

- Pipeline-level parallelism (PLP)
  - Multiple pipeline stages execute in parallel

- Instruction-level parallelism (ILP)
  - Multiple instructions execute in parallel
  - Examples: VLIW, In-Order SuperScalar, Out-of-Order SuperScalar

- Thread-level parallelism (TLP)
  - Multiple threads execute in parallel
  - Examples: SMT, Multicore, GPUs (as we will learn)